

SD8 — URL Shortener (the warm-up you finish fast and score on)

Oracle: ★★★ reported, often as the *first* SD prompt. Goal: clean 25-minute answer with numbers, then use the saved time on deep dives the interviewer picks. Tags: [hot path] · [async] · [control plane] · ? = follow-up + answer.

Index

0. Time budget
1. Why this is hard (it's mostly not — say what is)
2. Crux
3. Clarifying questions
4. FRs / NFRs
5. Back-of-envelope
6. High-level design
7. API
8. Data model
9. Deep dives (3)
10. Hygiene + ops
11. Pop-up questions
12. Trade-offs + defeaters
13. Wrap-up + card
14. References

§0 Time budget

Clarify 3 · FR/NFR/BoE 5 · HLD 6 · API+schema 4 · deep dives 8 · wrap 2 — **finish early on purpose.**

§1 Why this is hard

naive: auto-increment id → base62 → done

actually breaks at: nothing, for a while – the hard parts are read latency at scale, id generation without a single counter, and cache behavior for a zipfian read mix

- The honest framing (scores points): writes are trivial; **reads are 100–1000× writes** and long-tail
- Real problems: unique short code with no coordination bottleneck · 301 vs 302 (caching vs analytics) · hot keys · abuse (open-redirect/phishing)

§2 Crux

- Read path: code → long_url at p99 < 20 ms, cacheable to ~90%+ hit ratio
- Id generation that scales horizontally and doesn't leak business volume
- Say early: "redirects tolerate eventual consistency; creation can be strongly consistent — cheap win"

§3 Clarifying questions

Question	Assumed
Scale?	100M new/yr (~3/s write), 10k reads/s peak
Custom aliases?	yes, unique, first-come
Expiry / delete?	optional TTL; soft delete
Analytics?	click counts async; not real-time
Auth?	API keys per tenant; public redirect

§4 FRs / NFRs

- FR: create (random or custom alias, optional TTL) · redirect · delete · basic click stats
- NFR: redirect p99 < 20 ms, 99.99% · create p99 < 100 ms · no code collision ever · 5-year retention → ~500M rows

§5 Back-of-envelope

- 500M urls × ~500 B ≈ 250 GB — **one Postgres + replicas holds this**; sharding is optional, say so
- Codes: base62, length 7 → $62^7 \approx 3.5 \times 10^{12} \gg 5 \times 10^8$ — collision-free headroom
- 10k reads/s, zipfian → cache 20% of keys ≈ 90%+ hits → DB sees ~1k reads/s → boring (good)
- Bandwidth: 10k/s × ~500 B response ≈ 5 MB/s — CDN optional, edge cache helps tail latency

§6 High-level design

```
[hot path] GET /{code} → LB → redirect svc → local LRU (sd03) → Redis → Postgres replica
                                     ↳ 301/302 Location: long_url — click event → queue [async]
[hot path] POST /api/urls → create svc → id gen (range allocator) → Postgres primary
[async]   click consumer → aggregate counts (per code per hour) → analytics table
[control plane] safety: creation-time URL scanning + blacklist; kill switch per code
```

§7 API

```
POST /api/v1/urls {long_url, custom_alias?, ttl_days?} Idempotency-Key: uuid
→ 201 {code, short_url} 409 alias taken 422 blocked-url
GET /{code} → 302 Location: long_url (404 unknown, 410 expired/deleted)
DELETE /api/v1/urls/{code} → 204 (soft)
GET /api/v1/urls/{code}/stats → {clicks_total, by_day[...]}
```

? 301 or 302? → "302 (or 307): 301 is cached permanently by browsers → you lose analytics and can never remap the code. If pure redirect performance mattered more than control, 301 + long CDN TTL."

§8 Data model

```
CREATE TABLE urls (
  code          varchar(10) PRIMARY KEY,      -- base62
  long_url      text NOT NULL,
  tenant_id     uuid,
  created_at    timestampz NOT NULL DEFAULT now(),
  expires_at    timestampz,
  deleted       boolean NOT NULL DEFAULT false
);
CREATE UNIQUE INDEX urls_alias ON urls (code); -- custom aliases share the namespace

CREATE TABLE id_ranges (                    -- range allocator for id generation
  range_start bigint PRIMARY KEY,
  leased_to   text NOT NULL,
  leased_at   timestampz NOT NULL
);

CREATE TABLE clicks_hourly (
  code varchar(10), hour timestampz, clicks bigint,
  PRIMARY KEY (code, hour)
);
```

§9 Deep dives

9.1 Id generation (the standard follow-up)

- Options: hash the URL (dedupe-friendly, collision handling needed) · random + retry (simple; retries rise as space fills) · **counter + base62** (no collisions; the counter must scale)
- Pick: central counter → **range allocator**: each app node leases a block of 100k ids from one DB row; encodes to base62; obfuscate (XOR/bit-mix) so codes don't reveal volume

```
ALPHABET = "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"
```

```
def encode62(n):
    if n == 0:
        return ALPHABET[0]
    out = []
    while n:
        n, r = divmod(n, 62)
        out.append(ALPHABET[r])
    return "".join(reversed(out))

class IdBlock:
    """Lease [start, start+size) from the DB; refill when exhausted."""
    def __init__(self, db, node_id, size=100_000):
        self.db, self.node_id, self.size = db, node_id, size
        self.next = self.end = 0

    def next_id(self):
        if self.next >= self.end:
            self.next = self.db.one(
                """INSERT INTO id_ranges (range_start, leased_to, leased_at)
                VALUES ((SELECT coalesce(max(range_start),0)+%s FROM id_ranges), %s, now())
                RETURNING range_start""", (self.size, self.node_id)).range_start
            self.end = self.next + self.size
            n, self.next = self.next, self.next + 1
        return n

# Node crash wastes the rest of its block – acceptable (62^7 headroom). Say it.
```

```
public final class Encoder {
    private static final char[] A =
"0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ".toCharArray();

    public static String encode62(long n) {
        if (n == 0) return "0";
        StringBuilder sb = new StringBuilder();
        for (; n > 0; n /= 62) sb.append(A[(int) (n % 62)]);
        return sb.reverse().toString();
    }
}

// Range lease mirrors the Python: one conditional INSERT ... RETURNING per 100k ids
// -> the counter row is touched 3/s ÷ 100k ≈ never; no bottleneck, no Snowflake needed.
```

? Why not UUID/Snowflake? → "UUID is 22+ chars — defeats 'short'. Snowflake works but ids leak timestamps and it's more machinery than one leased-range row for 3 writes/s."

9.2 Cache strategy (where the QPS goes)

- Two tiers: in-process LRU (sd03, ~1 ms) → Redis (~2 ms) → replica (~5–10 ms); cache-aside, TTL 1 h + jitter
- Negative caching: unknown codes (bot scans) cached as 404 for 5 min — else a scan storm is a DB storm
- Delete/remap: write-through delete → Redis DEL + pub/sub invalidate to node caches (bounded staleness = local TTL)
- Hot key (viral link): per-node LRU absorbs it by design — that's the argument for the local tier

9.3 Abuse + safety (differentiator section)

- Creation-time: blocklist + reputation API check; 422 on known-bad
- Serve-time: kill switch (deleted flag → 410, cache invalidated) for takedowns
- Rate limit creation per key (sd01); CAPTCHA on anonymous tier
- Open-redirect phishing is *the* abuse of shorteners — saying it unprompted signals production experience

§10 Hygiene + ops

- Metrics: redirect p99, cache hit ratio (page < 80%), 404 rate (scan detector), create errors, replica lag
- Ops: DB failover drill; cache cold-start (stampede → warm from access log top-k; `getOrLoad` singleflight from sd03)
- Data: clicks are append-heavy → partition `clicks_hourly` monthly; archive raw events to object storage

§11 Pop-up questions

- *10× reads (100k/s)?* → "CDN in front of redirects (short TTL + purge hooks), more replicas, same design. The DB still sees ~cache-miss traffic only."
- *Multi-region?* → "Reads: replicate everywhere, serve local. Writes: single home region (3/s!) or per-region code prefix to avoid write conflicts."
- *Analytics real-time?* → "Click events → stream → 1-min windows; the redirect path never waits on analytics."
- *Custom alias contention?* → "Unique index is the arbiter; 409 on conflict. No reservation flow in v1."
- *Expiry?* → "Lazy on read (410 + tombstone) plus a daily sweep; exactly the sd03 TTL pattern."

§12 Trade-offs + defeaters

Decision	Chosen	Alternative	Flips when
Ids	leased ranges + base62	random + retry	write rate >> 10k/s (rare here)
Redirect	302	301 + CDN	analytics don't matter
Store	Postgres + replicas	KV store (sd11)	> 10 TB or multi-region writes
Dedupe	none (each create new)	hash-based	storage cost of duplicates matters

- Defeaters: "hash and truncate, collisions are rare" (birthday paradox at scale — handle or avoid) · "301 everywhere" (permanent cache = no control) · "shard day one" (250 GB is one database)

\$13 Wrap-up + card

- Pitch: leased-range base62 ids; two-tier cache in front of Postgres replicas; 302 with async click events; abuse controls at create and serve time
- 5 numbers: 3/s writes · 10k/s reads · $62^7 \approx 3.5\text{T}$ codes · 250 GB / 5 y · 90% cache hit
- 3 failures: cache stampede (singleflight) · viral hot key (local LRU) · scan storm (negative cache)
- 3 defeaters: truncate-hash · 301-everywhere · premature sharding
- Done when: full answer ≤ 25 min with BoE from memory; encode62 + range lease written cold

\$14 References

- Alex Xu vol 1 ch. 8; Hello Interview URL shortener write-up
- bit.ly engineering talks (hot-key + analytics separation)